

Course design.

Basics	Systems	Scaling laws	Data	Algorithms
Tokenization	Kernels	Scaling sequence	Evaluation	Supervised fine-tuning
Architecture	Parallelism	Model complexity	Curriculum	Reinforcement learning
Loss function	Quantization	Loss metrics	Transformers	Synthetic data
Optimizer	Activation checks	Parameter form	Filtering	Verifications
Learning rate	CPU offloading		Deduplication	
	Inference		Mixing	

Byte-pair Encoding (BPE) tokenizer

use bytes instead of tokens.

⇒ directly token-free approach.

no Models Architecture

edge byte

not scaled up yet for to transformers

Transformer variables.

* Activation functions: ReLU, SwiGLU.

* pos encoding: sinusoidal, RoPE

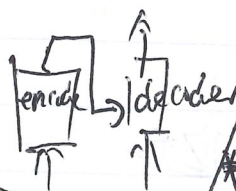
* normalization: layer norm / RMSNorm

* placement of norm: pre-norm / vs / post-norm

* MLP / FNN: dense / sparse / MoE

* Attention: Full / self / flash /paged / Radix - attention

* low-dim: MLA / GQA / flash: * hybrid transform: Hyena / Trans-Vnext



Train:

* optimizer: Adam / SWAP

* LR Rate schedule:

* Batch size = critical batch (cosine-ws)

* regulation = dropout / weight decay

* hyperparameter: grid search

Hardware / chip / GPU / grid / blocks / smart / threads
 map
 systems: $\left\{ \begin{array}{l} \text{memory} \rightleftharpoons \text{compute} \\ \text{(DRAM)} \quad \uparrow \quad \text{(SRAM + GPU)} \end{array} \right.$

kernel op / overlaid fusion / ~~tiny~~ memory edge

parallelism: 8 flows $\left\{ \begin{array}{l} \text{data movement across GPUs} \\ \text{use collective communications} \\ \text{shared parameter/activation/gradients} \end{array} \right.$
 How to split communications

inference $\left\{ \begin{array}{l} \text{process all at once} \\ \text{prefill} \rightarrow \text{compute-bound} \\ \text{decode} \rightarrow \text{memory-bound} \\ \text{auto-regressive} \end{array} \right.$

for: $\left\{ \begin{array}{l} \text{RL} \\ \text{test} \\ \text{eval} \end{array} \right.$

$\Rightarrow$ use cheaper model
 $\Rightarrow$ model pruning / quantization / distillation
 $\Rightarrow$ speculative decoding
 $\Rightarrow$ system opti: KV cache, batchy

Scaling laws $\Rightarrow$

Model Size $\times 20 =$ Num of tokens for training: (1.4B models requires 28B training tokens)

Assignment: implement baby version